Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Операционные системы и системное программирование

ОТЧЕТ
по лабораторной работе № 1
Тема: «Знакомство с Linux/Unix и средой программирования. POSIX-совместимая файловая система»

Выполнил:
ст. гр. 450502 Климова А. А.

Проверил:
Благиных В. Д.

Минск, 2026

1 ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ

Изучить POSIX-совместимую файловую систему, команды и утилиты оболочки.

Разработать программу dirwalk, сканирующую файловую систему и выводящую в stdout информацию в соответствии с опциями программы.

Формат вывода аналогичен формату вывода утилиты find. dirwalk [dir] [options] – dir начальный каталог.

Опции options:

-l – только символические ссылки (-type l)

-d – только каталоги (-type d)

-f – только файлы (-type f)

-s – сортировка по LC_COLLOCATE (-type f)

Опции могут быть указаны как перед каталогом, так и после. Опции могут быть указаны как раздельно, так и вместе (-l -d, -ld). Если опции ldf опущены, выводятся каталоги, файлы и ссылки.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Описание алгоритма выполнения работы

Программа dirwalk выполняет рекурсивный обход файловой системы, начиная с заданного каталога (по умолчанию ./), и выводит пути объектов в stdout в формате, аналогичном find (один путь на строку).

Алгоритм работы:

1) Разбор аргументов командной строки:
   - поддерживаются опции -l, -d, -f, -s;
   - опции можно указывать до и после каталога, а также объединять (например, -ld);
   - если фильтры -l/-d/-f не заданы, выводятся каталоги, регулярные файлы и симлинки.

2) При наличии -s включается локаль LC_COLLATE для корректной сортировки строк.

3) Выполняется рекурсивный обход дерева каталогов:
   - для каждого пути вызывается lstat();
   - тип объекта определяется через S_ISLNK/S_ISDIR/S_ISREG;
   - если объект подходит под фильтр, путь либо печатается сразу, либо сохраняется в динамический массив (режим -s).

4) Для режима -s после завершения обхода список путей сортируется по strcoll() и выводится.

5) Освобождается вся выделенная память, программа завершается с кодом результата.

Программа реализована на переносимых POSIX-вызовах (opendir/readdir/closedir/lstat/getopt), без Linux-специфичных расширений.

2.2 Описание основных функций и модулей

2.2.1 Модуль options (include/options.h, src/options.c)

- print_usage(const char *prog_name, FILE *out)

  Выводит краткую справку по синтаксису и опциям.
- parse_options(int argc, char *argv[], DirwalkOptions *opts)

  Разбирает аргументы через getopt(3), определяет стартовый каталог и активные флаги.

  В opts записываются:
  - want_links (фильтр симлинков),
  - want_dirs (фильтр каталогов),
  - want_files (фильтр регулярных файлов),
  - sort_output (сортировка),
  - start_dir (стартовый путь)..

2.2.2 Модуль strlist (include/strlist.h, src/strlist.c)

Структура StrList:

- char **items — массив строк (путей),
- size_t size — текущее число элементов,
- size_t capacity — выделенная емкость.

Функции:

- strlist_init() — инициализация пустого массива.
- strlist_push() — добавление пути:

при нехватке места емкость увеличивается (обычно в 2 раза), затем строка копируется в собственную память.

- strlist_sort_locale() — сортировка через qsort() + strcoll().
- strlist_free() — освобождение памяти всех строк и массива.

2.2.3 Модуль walker (include/walker.h, src/walker.c)

Основная функция:

- walk_tree(const char *root, const DirwalkOptions *opts, StrList *output)

  Выполняет обход дерева от root.

  Если output == NULL, подходящие пути печатаются сразу.

  Если output != NULL, подходящие пути накапливаются в списке (для -s).

Внутренние функции:

- join_path() — формирование полного пути дочернего элемента.
- matches_type() — проверка соответствия типа файла выбранным фильтрам.
- emit_path() — печать пути или добавление в список.
- walk_node() — рекурсивный обход каталога, обработка ошибок доступа.

2.2.4 Главный модуль (src/main.c)

Функция main():

1) вызывает parse_options();
2) инициализирует StrList;
3) при -s включает setlocale(LC_COLLATE, "");
4) запускает walk_tree();
5) при -s сортирует и выводит накопленные пути;
6) освобождает память и возвращает код завершения.

2.3 Функциональная структура проекта

Проект разделен на интерфейсы (.h) и реализации (.c):

- include/options.h: интерфейс парсинга опций;
- include/strlist.h: интерфейс динамического массива строк;
- include/walker.h: интерфейс обхода файловой системы;

- src/options.c: реализация парсинга аргументов;
- src/strlist.c: реализация хранения/сортировки путей;
- src/walker.c: реализация рекурсивного обхода;
- src/main.c: точка входа и связывание модулей;
- Makefile: сборка исполняемого файла dirwalk;
- README.md: краткая инструкция и примеры.

Такое разбиение упрощает сопровождение: логика опций, обхода и хранения данных изолирована по модулям.

3 ОПИСАНИЕ ПОРЯДКА СБОРКИ

Сборка выполняется с помощью Makefile.
Команды:
1) Полная пересборка: make clean && make;
2) Обычная сборка: make;
3) Очистка артефактов: make clean.
Результат сборки: исполняемый файл ./dirwalk.

4 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

Синтаксис:
dirwalk [dir] [options]
Опции:
-l только симлинки
-d только каталоги
-f только регулярные файлы
-s сортировка по LC_COLLATE

Проверка корректности (сравнение с find + wc). Попарные значения должны совпасть.

```
avelinaklimova@MacBook-Air-Avelina lab1 % ./dirwalk /private/etc -fdl | wc -l
dirwalk: cannot open directory '/private/etc/cups/certs': Permission denied
     267
avelinaklimova@MacBook-Air-Avelina lab1 % ./dirwalk /private/etc -d | wc -l
dirwalk: cannot open directory '/private/etc/cups/certs': Permission denied
      33
avelinaklimova@MacBook-Air-Avelina lab1 % ./dirwalk /private/etc -f | wc -l
dirwalk: cannot open directory '/private/etc/cups/certs': Permission denied
     230
avelinaklimova@MacBook-Air-Avelina lab1 % ./dirwalk /private/etc -l | wc -l
dirwalk: cannot open directory '/private/etc/cups/certs': Permission denied
       4
avelinaklimova@MacBook-Air-Avelina lab1 % find /private/etc \( -type f -o
-type d -o -type l \) | wc -l
find: /private/etc/cups/certs: Permission denied
     267
avelinaklimova@MacBook-Air-Avelina lab1 % find /private/etc -type d | wc -l
find: /private/etc/cups/certs: Permission denied
      33
avelinaklimova@MacBook-Air-Avelina lab1 % find /private/etc -type f | wc -l
find: /private/etc/cups/certs: Permission denied
     230
avelinaklimova@MacBook-Air-Avelina lab1 % find /private/etc -type l | wc -l
find: /private/etc/cups/certs: Permission denied
       4
```

Рисунок 1 – Сравнение результатов поиска find и dirwalk

```
avelinaklimova@MacBook-Air-Avelina lab1 % ./dirwalk /private/etc -s | head -n
10
dirwalk: cannot open directory '/private/etc/cups/certs': Permission denied
/private/etc
/private/etc/afpovertcp.cfg
/private/etc/aliases
/private/etc/aliases.db
/private/etc/apache2
/private/etc/apache2/extra
/private/etc/apache2/extra/httpd-autoindex.conf
/private/etc/apache2/extra/httpd-dav.conf
/private/etc/apache2/extra/httpd-default.conf
/private/etc/apache2/extra/httpd-info.conf
avelinaklimova@MacBook-Air-Avelina lab1 % find /private/etc \( -type f -o
-type d -o -type l \) | sort | head -n 10
find: /private/etc/cups/certs: Permission denied
/private/etc
/private/etc/afpovertcp.cfg
/private/etc/aliases
/private/etc/aliases.db
/private/etc/apache2
/private/etc/apache2/extra
/private/etc/apache2/extra/httpd-autoindex.conf
/private/etc/apache2/extra/httpd-dav.conf
/private/etc/apache2/extra/httpd-default.conf
/private/etc/apache2/extra/httpd-info.conf
```

Рисунок 2 – Сравнение результатов сортировки find и dirwalk

Примечание для macOS:

В большинстве систем /etc является симлинком на /private/etc, поэтому для демонстрации всех трех типов файлов корректнее использовать именно /private/etc.

5 ВЫВОД

При выполнении лабораторной работы была изучена работа с файлами в операционной системе Linux.

В рамках выполнения работы была создана программа, которая анализирует файлы и выводит информацию в соответствии с заданными параметрами. Программа поддерживает различные опции, позволяющие гибко настраивать процесс сканирования и формат вывода данных.

ПРИЛОЖЕНИЕ А

Программный код

```
Файл options.h
#ifndef OPTIONS_H
#define OPTIONS_H

#include <stdio.h>

typedef struct {
    int want_links;
    int want_dirs;
    int want_files;
    int sort_output;
    const char *start_dir;
} DirwalkOptions;

int parse_options(int argc, char *argv[], DirwalkOptions *opts);
void print_usage(const char *prog_name, FILE *out);

#endif

Файл options.c
#include "options.h"

#include <getopt.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void print_usage(const char *prog_name, FILE *out)
{
    fprintf(out, "Usage: %s [dir] [options]\n", prog_name);
    fprintf(out, "Options:\n");
    fprintf(out, "  -l    only symbolic links\n");
    fprintf(out, "  -d    only directories\n");
    fprintf(out, "  -f    only regular files\n");
    fprintf(out, "  -s    sort output by LC_COLLATE\n");
}

int parse_options(int argc, char *argv[], DirwalkOptions *opts)
{
    int i, c;
    int after_dashdash = 0;
    int option_argc = 1;
    const char *start_dir = NULL;
    char **option_argv;
```

```
    opts->want_links = 0;
    opts->want_dirs = 0;
    opts->want_files = 0;
    opts->sort_output = 0;
    opts->start_dir = "./";

    option_argv = malloc((size_t)(argc + 1) * sizeof(*option_argv));
    if (!option_argv) {
        fprintf(stderr, "Error: out of memory while parsing
options.\n");
        return -1;
    }

    option_argv[0] = argv[0];

    for (i = 1; i < argc; ++i) {
        const char *arg = argv[i];

        if (!after_dashdash && strcmp(arg, "--") == 0) {
            after_dashdash = 1;
            continue;
        }

        if (!after_dashdash && arg[0] == '-' && arg[1] != '\0') {
            option_argv[option_argc++] = argv[i];
            continue;
        }

        if (!start_dir) {
            start_dir = arg;
        } else {
            fprintf(stderr, "Error: only one start directory is
allowed.\n");
            free(option_argv);
            return -1;
        }
    }

    opterr = 0;
    optind = 1;
    while ((c = getopt(option_argc, option_argv, "ldfs")) != -1) {
        switch (c) {
        case 'l':
            opts->want_links = 1;
            break;
        case 'd':
            opts->want_dirs = 1;
            break;
        case 'f':
            opts->want_files = 1;
```

```
            break;
        case 's':
            opts->sort_output = 1;
            break;
        default:
            fprintf(stderr, "Error: unknown option '-%c'.\n", optopt ?
optopt : '?');
            print_usage(argv[0], stderr);
            free(option_argv);
            return -1;
        }
    }

    if (start_dir) {
        opts->start_dir = start_dir;
    }

    free(option_argv);
    return 0;
}
```

Файл strlist.h

```
#ifndef STRLIST_H
#define STRLIST_H

#include <stddef.h>

typedef struct {
    char **items;
    size_t size;
    size_t capacity;
} StrList;

void strlist_init(StrList *list);
void strlist_free(StrList *list);
int strlist_push(StrList *list, const char *value);
void strlist_sort_locale(StrList *list);

#endif
```

Файл strlist.c

```
#include "strlist.h"

#include <stdlib.h>
#include <string.h>

static int compare_locale(const void *lhs, const void *rhs)
{
    const char *a = *(const char *const *)lhs;
    const char *b = *(const char *const *)rhs;
    return strcoll(a, b);
```

```
}

void strlist_init(StrList *list)
{
    list->items = NULL;
    list->size = 0;
    list->capacity = 0;
}

void strlist_free(StrList *list)
{
    size_t i;
    for (i = 0; i < list->size; ++i) {
        free(list->items[i]);
    }
    free(list->items);
    list->items = NULL;
    list->size = 0;
    list->capacity = 0;
}

int strlist_push(StrList *list, const char *value)
{
    char **new_items;
    char *copy;

    if (list->size == list->capacity) {
        size_t new_capacity = list->capacity ? list->capacity * 2 :
64;
        new_items = realloc(list->items, new_capacity *
sizeof(*new_items));
        if (!new_items) {
            return -1;
        }
        list->items = new_items;
        list->capacity = new_capacity;
    }

    copy = malloc(strlen(value) + 1);
    if (!copy) {
        return -1;
    }
    strcpy(copy, value);
    list->items[list->size++] = copy;
    return 0;
}

void strlist_sort_locale(StrList *list)
{
    if (list->size > 1) {
```

```
        qsort(list->items, list->size, sizeof(*list->items),
compare_locale);
    }
}
```

Файл walker.h

```
#ifndef WALKER_H
#define WALKER_H

#include "options.h"
#include "strlist.h"

int walk_tree(const char *root, const DirwalkOptions *opts, StrList
*output);

#endif
```

Файл walker.c

```
#include "walker.h"

#include <dirent.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>

typedef struct {
    const DirwalkOptions *opts;
    StrList *output;
    int had_access_errors;
    int output_failed;
} WalkState;

static char *join_path(const char *base, const char *name)
{
    size_t base_len = strlen(base);
    size_t name_len = strlen(name);
    int needs_slash = (base_len > 0 && base[base_len - 1] != '/');
    size_t total_len = base_len + (size_t)needs_slash + name_len + 1;
    char *result = malloc(total_len);

    if (!result) {
        return NULL;
    }

    memcpy(result, base, base_len);
    if (needs_slash) {
        result[base_len] = '/';
        memcpy(result + base_len + 1, name, name_len);
```

```
        result[base_len + 1 + name_len] = '\0';
    } else {
        memcpy(result + base_len, name, name_len);
        result[base_len + name_len] = '\0';
    }

    return result;
}

static int matches_type(mode_t mode, const DirwalkOptions *opts)
{
    int no_filter = !opts->want_links && !opts->want_dirs &&
!opts->want_files;

    if (no_filter) {
        return S_ISLNK(mode) || S_ISDIR(mode) || S_ISREG(mode);
    }

    if (opts->want_links && S_ISLNK(mode)) {
        return 1;
    }
    if (opts->want_dirs && S_ISDIR(mode)) {
        return 1;
    }
    if (opts->want_files && S_ISREG(mode)) {
        return 1;
    }
    return 0;
}

static int emit_path(const char *path, WalkState *state)
{
    if (state->output) {
        if (strlist_push(state->output, path) != 0) {
            fprintf(stderr, "dirwalk: out of memory\n");
            return -1;
        }
        return 0;
    }

    if (puts(path) == EOF) {
        state->output_failed = 1;
        return -1;
    }
    return 0;
}

static int walk_node(const char *path, WalkState *state)
{
    struct stat st;
```

```
    DIR *dir;
    struct dirent *entry;

    if (lstat(path, &st) ≠ 0) {
        fprintf(stderr, "dirwalk: cannot access '%s': %s\n", path,
strerror(errno));
        state→had_access_errors = 1;
        return 0;
    }

    if (matches_type(st.st_mode, state→opts)) {
        if (emit_path(path, state) ≠ 0) {
            return -1;
        }
    }

    if (!S_ISDIR(st.st_mode)) {
        return 0;
    }

    dir = opendir(path);
    if (!dir) {
        fprintf(stderr, "dirwalk: cannot open directory '%s': %s\n",
path, strerror(errno));
        state→had_access_errors = 1;
        return 0;
    }

    while ((entry = readdir(dir)) ≠ NULL) {
        char *child_path;
        int rc;

        if (strcmp(entry→d_name, ".") == 0 || strcmp(entry→d_name,
"..") == 0) {
            continue;
        }

        child_path = join_path(path, entry→d_name);
        if (!child_path) {
            fprintf(stderr, "dirwalk: out of memory\n");
            closedir(dir);
            return -1;
        }

        rc = walk_node(child_path, state);
        free(child_path);
        if (rc ≠ 0) {
            closedir(dir);
            return rc;
        }
```

```
    }

    if (closedir(dir) ≠ 0) {
        fprintf(stderr, "dirwalk: cannot close directory '%s': %s\n",
path, strerror(errno));
        state→had_access_errors = 1;
    }

    return 0;
}

int walk_tree(const char *root, const DirwalkOptions *opts, StrList
*output)
{
    WalkState state;
    int rc;

    state.opts = opts;
    state.output = output;
    state.had_access_errors = 0;
    state.output_failed = 0;

    rc = walk_node(root, &state);
    if (rc ≠ 0) {
        return -1;
    }
    if (state.output_failed) {
        return -1;
    }
    return state.had_access_errors ? 1 : 0;
}

Файл main.c
#include "options.h"
#include "strlist.h"
#include "walker.h"

#include <locale.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    DirwalkOptions opts;
    StrList paths;
    int walk_result;
    size_t i;

    if (parse_options(argc, argv, &opts) ≠ 0) {
        return EXIT_FAILURE;
```

```
    }

    strlist_init(&paths);
    if (opts.sort_output) {
        if (!setlocale(LC_COLLATE, "")) {
            fprintf(stderr, "dirwalk: warning: cannot apply locale
collation\n");
        }
    }

    walk_result = walk_tree(opts.start_dir, &opts, opts.sort_output ?
&paths : NULL);
    if (walk_result == -1) {
        strlist_free(&paths);
        return EXIT_FAILURE;
    }

    if (opts.sort_output) {
        strlist_sort_locale(&paths);
        for (i = 0; i < paths.size; ++i) {
            if (puts(paths.items[i]) == EOF) {
                strlist_free(&paths);
                return EXIT_FAILURE;
            }
        }
    }

    strlist_free(&paths);
    return walk_result == 0 ? EXIT_SUCCESS : EXIT_FAILURE;
}
```